

# Strings

## Conceptos y Estructuras Fundamentales

Luis Santiago Re

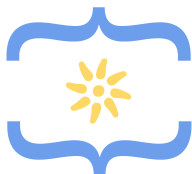
Universidad Tecnológica Nacional - Santa Fe

Training Camp 2026



# Gracias Sponsors!

Organiza



Diamond Sponsor



Facultad de  
**INFORMÁTICA**



UNIVERSIDAD  
NACIONAL  
DE LA PLATA

# Outline

- ➊ Introducción
- ➋ Trie
- ➌ Hashing
- ➍ Función Z
- ➎ Resumen y Consultas

# Outline

- ➊ Introducción
- ➋ Trie
- ➌ Hashing
- ➍ Función Z
- ➎ Resumen y Consultas

# Definiciones Básicas

- string: arreglo o cadena de caracteres.
  - Ejemplo: "banana"
- substring: secuencia no vacía y consecutiva de caracteres dentro de un string.
  - Ejemplos de "banana": "bana", "nan", "ana"
  - Un string de largo  $n$  tiene  $n \times (n + 1)/2$  substrings posibles.
- prefijo: substring que inicia en el primer caracter del string.
  - Ejemplos: "b", "ban", "banana"
- sufijo: substring que termina en el último caracter del string.
  - Ejemplos: "a", "nana", "banana"
- alfabeto ( $\Sigma$ ): conjunto de caracteres que pueden aparecer.
  - Usualmente el conjunto de 26 letras minúsculas del abecedario inglés.

- Declaración  $\rightarrow$  `string s = "hola mundo";`
- Acceso a índice  $i \rightarrow$  `s[i];`
- Largo del string  $\rightarrow$  `s.size();` o `s.length();`
- Concatenar caracter  $c \rightarrow$  `s += c;` o `s.push_back(c);`
- Concatenar string  $t \rightarrow$  `s += t;`
- Substring desde el índice `idx` de largo `len`  $\rightarrow$  `s.substr(idx, len);`

# Outline

- 1 Introducción
- 2 **Trie**
- 3 Hashing
- 4 Función Z
- 5 Resumen y Consultas

# Problema 1: Búsqueda de Prefijos

## Enunciado

Se tiene un diccionario de  $N$  palabras. Luego, nos dan  $Q$  queries, cada una con una palabra  $P$ , y queremos saber si existe alguna palabra en el diccionario que tenga a  $P$  como prefijo.

# Problema 1: Búsqueda de Prefijos

## Enunciado

Se tiene un diccionario de  $N$  palabras. Luego, nos dan  $Q$  queries, cada una con una palabra  $P$ , y queremos saber si existe alguna palabra en el diccionario que tenga a  $P$  como prefijo.

Solución bruta:

- Para cada query, iteramos por las  $N$  palabras del diccionario y revisamos si  $P$  es prefijo de alguna comparando caracter a caracter.

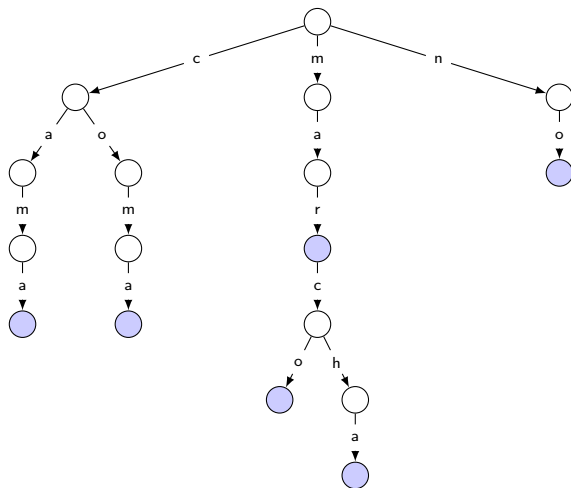
# Definición de Trie

Un Trie (o Prefix Tree) es un árbol que representa un conjunto de strings:

- La raíz representa el string vacío.
- Cada arista representa un caracter. Recorrer un camino desde la raíz construye un prefijo.
- Cada nodo representa un prefijo de alguna palabra del conjunto.
- Los nodos donde termina una palabra completa se marcan como terminales.

La complejidad para insertar o buscar una palabra de largo  $L$  se reduce a  $\mathcal{O}(L \times \log |\Sigma|)$  o  $\mathcal{O}(L)$  dependiendo de la implementación.

# Trie - Ejemplo



Ejemplo de trie para el diccionario "cama", "coma", "mar", "marco", "marcha", "no". Los nodos azules son terminales.

# Código: Estructura Trie General

## Implementación en C++

```
struct TrieNode {
    map<char, int> hijos;
    bool term = false;
};

vector<TrieNode> trie(1);

void insertar(const string& s) {
    int u = 0; // arrancamos en la raiz
    for (char c : s) {
        if (!trie[u].hijos.count(c)) {
            trie[u].hijos[c] = trie.size();
            trie.push_back(TrieNode());
        }
        u = trie[u].hijos[c];
    }
    trie[u].term = true;
}
```

# Problema 1: Solución con Trie

Solución con Trie:

- Complejidad por query:  $\mathcal{O}(|P|)$ . ¡Muchísimo más rápido!

## Problema 2: Prefijos y Borrado

### Enunciado

Mantenemos un conjunto de palabras (inicialmente vacío). Queremos soportar 3 tipos de queries:

- 1 Agregar la palabra  $P$  al conjunto.
- 2 Borrar la palabra  $P$  del conjunto (se asegura que existe).
- 3 Dada una palabra  $P$ , contar cuántas palabras del conjunto actual tienen a  $P$  como prefijo.

## Problema 2: Prefijos y Borrado

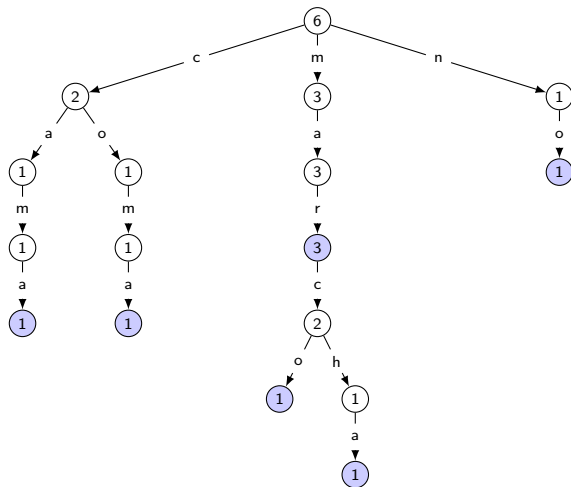
### Enunciado

Mantenemos un conjunto de palabras (inicialmente vacío). Queremos soportar 3 tipos de queries:

- 1 Agregar la palabra  $P$  al conjunto.
- 2 Borrar la palabra  $P$  del conjunto (se asegura que existe).
- 3 Dada una palabra  $P$ , contar cuántas palabras del conjunto actual tienen a  $P$  como prefijo.

Solución: Además de las transiciones, en cada nodo del Trie guardaremos una variable `cnt` que indique cuántas palabras pasan por ese nodo. Cuando insertamos, sumamos 1 a todos los nodos del camino. Cuando borramos, restamos 1. Para buscar cuántas tienen a  $P$  de prefijo, bajamos por el Trie usando los caracteres de  $P$ ; la respuesta será el `cnt` del nodo donde terminamos.

# Trie - Dibujo Explicación



Estado del Trie tras insertar "cama", "coma", "mar", "marco", "marcha", "no". Cada nodo indica cuántas palabras lo visitaron.

## Modificación en C++

```
struct TrieNode {
    map<char, int> hijos;
    int cnt = 0; // Cantidad de palabras que pasan por aca
};
// ...
void insertar(const string& s) {
    int u = 0;
    trie[u].cnt++;
    for (char c : s) {
        if (!trie[u].hijos.count(c)) {
            trie[u].hijos[c] = trie.size();
            trie.push_back(TrieNode());
        }
        u = trie[u].hijos[c];
        trie[u].cnt++;
    }
}
```

Para borrar, hacemos un recorrido similar pero haciendo `cnt--`.

## Problema 3: Vasiliy's Multiset

### CF 706D - Vasiliy's Multiset

Dado un multiset inicialmente con un único valor 0, debemos procesar  $Q \leq 2 \times 10^5$  queries:

- +  $x$ : agregar el número  $x$ .
- -  $x$ : borrar una ocurrencia del número  $x$ .
- ?  $x$ : encontrar un elemento  $y$  en el multiset tal que  $x \oplus y$  (XOR) sea máximo.

# Problema 3: Vasiliy's Multiset

## CF 706D - Vasiliy's Multiset

Dado un multiset inicialmente con un único valor 0, debemos procesar  $Q \leq 2 \times 10^5$  queries:

- $+$   $x$ : agregar el número  $x$ .
- $-$   $x$ : borrar una ocurrencia del número  $x$ .
- $?$   $x$ : encontrar un elemento  $y$  en el multiset tal que  $x \oplus y$  (XOR) sea máximo.

Input de ejemplo:

+ 8  
+ 9  
+ 11  
? 3  
- 8  
? 3

Output:

11  
10

## Problema 3: Solución

Idea (Binary Trie): Podemos ver a los números como strings de 30 bits (0s y 1s). Insertamos los números en un Trie binario. Para la query ?  $x$ , recorremos el Trie desde la raíz intentando tomar siempre la rama que tiene el bit opuesto al de  $x$  (para maximizar el XOR en ese bit). Si ese nodo existe y tiene `cnt > 0`, bajamos por ahí. Si no, bajamos por la rama que tiene el mismo bit.

# Outline

- 1 Introducción
- 2 Trie
- 3 Hashing**
- 4 Función Z
- 5 Resumen y Consultas

## Introducción

- La idea de String Hashing es convertir un string a un número (su "hash") para comparar strings enteros en  $\mathcal{O}(1)$  en lugar de  $\mathcal{O}(L)$ .
- Un hash común es el hash polinomial: elegimos una base  $B > |\Sigma|$  y evaluamos el string como si fuera un polinomio:

$$H(S) = (s_0 \times B^0 + s_1 \times B^1 + \dots + s_{L-1} \times B^{L-1}) \pmod{M}$$

- Si  $H(S) = H(T)$ , asumimos que  $S = T$ . Si  $H(S) \neq H(T)$ , sabemos seguro que  $S \neq T$ .
- Si calculamos los hashes de todos los prefijos, el hash de un subrango  $[i, j]$  se puede calcular en  $\mathcal{O}(1)$ :

$$H(S[i \dots j]) = (H(S[0 \dots j]) - H(S[0 \dots i-1])) \times B^{-i} \pmod{M}$$

# Hashing: Módulo y Colisiones

- Los valores del polinomio pueden ser enormes, así que usamos un módulo primo  $M$  grande (ej.  $10^9 + 7$ ).
- Colisión: Cuando dos strings distintos tienen el mismo hash.
- La probabilidad de colisión es aproximadamente  $\frac{1}{M}$ .
- Para evitar que nos hackeen la solución con colisiones, se recomienda usar doble hash (dos módulos primos distintos  $M_1, M_2$  y bases distintas).
- Si  $H_1(S) = H_1(T)$  y  $H_2(S) = H_2(T)$ , la probabilidad de colisión se vuelve infinitesimal ( $\approx \frac{1}{M_1 \times M_2}$ ).
- Al extraer un subrango, usamos el inverso modular de la base respecto a  $M$  para dividir por  $B^i$ .

- Precálculo:  $\mathcal{O}(N)$  para calcular el hash de cada prefijo y precomputar las potencias de la base  $B^k \pmod{M}$ .
- Query:  $\mathcal{O}(1)$  para obtener el hash de cualquier substring.
- Mucha flexibilidad: se pueden comparar prefijos, sufijos, subrangos, concatenar hashes matemáticamente, etc.

Pueden encontrar código de referencia para doble hash en:

[github.com/.../hash.cpp](https://github.com/.../hash.cpp)

# Problema 1: String Matching

## Enunciado

Dado un texto  $T$  y un patrón  $P$ , encontrar el número de ocurrencias de  $P$  en  $T$ .

# Problema 1: String Matching

## Enunciado

Dado un texto  $T$  y un patrón  $P$ , encontrar el número de ocurrencias de  $P$  en  $T$ .

Solución bruta: Iteramos por cada posición inicial  $i$  en el texto  $T$  y comparamos el substring con el patrón. Complejidad:  $\mathcal{O}(|T| \times |P|)$ .

# Problema 1: String Matching

## Enunciado

Dado un texto  $T$  y un patrón  $P$ , encontrar el número de ocurrencias de  $P$  en  $T$ .

Solución bruta: Iteramos por cada posición inicial  $i$  en el texto  $T$  y comparamos el substring con el patrón. Complejidad:  $\mathcal{O}(|T| \times |P|)$ .

Solución con Hashing:

- Precalculamos los hashes de los prefijos de  $T$  en  $\mathcal{O}(|T|)$ .
- Calculamos el hash del patrón  $P$  en  $\mathcal{O}(|P|)$ .
- Para cada posición  $i$ , extraemos el hash del substring en  $\mathcal{O}(1)$  y verificamos si coincide con el de  $P$ .
- Complejidad final:  $\mathcal{O}(|T| + |P|)$ .

## Problema 2: Comparación Lexicográfica

### Enunciado

Queremos saber cuál de dos strings  $S$  y  $T$  (o substrings de éstos) es lexicográficamente menor.

## Problema 2: Comparación Lexicográfica

### Enunciado

Queremos saber cuál de dos strings  $S$  y  $T$  (o substrings de éstos) es lexicográficamente menor.

Solución bruta: Comparar caracter a caracter hasta encontrar una diferencia. Tarda  $\mathcal{O}(\min(|S|, |T|))$ .

## Problema 2: Comparación Lexicográfica

### Enunciado

Queremos saber cuál de dos strings  $S$  y  $T$  (o substrings de éstos) es lexicográficamente menor.

Solución bruta: Comparar caracter a caracter hasta encontrar una diferencia. Tarda  $\mathcal{O}(\min(|S|, |T|))$ .

Solución con Hashing:

- El prefijo común más largo (LCP) nos dice exactamente en qué índice difieren ambos strings por primera vez.
- Si usamos búsqueda binaria sobre la longitud del prefijo y verificamos igualdad con Hashing en  $\mathcal{O}(1)$ , encontramos el LCP en  $\mathcal{O}(\log(\min(|S|, |T|)))$ .
- Sabiendo el LCP, comparamos el siguiente caracter en  $\mathcal{O}(1)$ .
- Complejidad total:  $\mathcal{O}(\log(\min(|S|, |T|)))$  por cada comparación.

# Problema 3: Longest Repeating Substring

## CSES 2106 - Repeating Substring

Dado un string  $S$  de longitud  $N$ , encontrar el substring repetido más largo (un substring que aparece al menos dos veces sin importar si se solapan).

# Problema 3: Longest Repeating Substring

## CSES 2106 - Repeating Substring

Dado un string  $S$  de longitud  $N$ , encontrar el substring repetido más largo (un substring que aparece al menos dos veces sin importar si se solapan).

Solución:

- Hacemos búsqueda binaria sobre la respuesta (la longitud  $L$ ).
- Para un largo  $L$  fijo, iteramos sobre los  $N - L + 1$  substrings posibles de tamaño  $L$ .
- Obtenemos el hash de cada uno en  $\mathcal{O}(1)$  y los guardamos (ej. en un `std::set`, `unordered_set` o los ordenamos en un vector).
- Si encontramos dos hashes iguales, significa que existe un substring repetido de tamaño  $L$ , ajustamos nuestra búsqueda binaria.
- Complejidad:  $\mathcal{O}(N \log^2 N)$  (o  $\mathcal{O}(N \log N)$ ).

# Problema 4: Palindrome Queries

## CSES 2420 - Palindrome Queries

Dado un string  $S$  de tamaño  $N$ , debemos soportar:

- Cambiar el caracter  $S[k]$  por  $x$ .
- Chequear si un subrango  $S[a \dots b]$  es un palíndromo.

# Problema 4: Palindrome Queries

## CSES 2420 - Palindrome Queries

Dado un string  $S$  de tamaño  $N$ , debemos soportar:

- Cambiar el caracter  $S[k]$  por  $x$ .
- Chequear si un subrango  $S[a \dots b]$  es un palíndromo.

Solución:

- Un string es palíndromo si se lee igual de izquierda a derecha que de derecha a izquierda.
- Mantenemos dos Segment Trees (o Fenwick Trees): uno para el hash del string normal y otro para el hash del string reverso.
- Actualizar se puede hacer en  $\mathcal{O}(\log N)$ .
- Consultar si un rango es palíndromo requiere obtener el hash del rango normal y compararlo con el rango equivalente en el reverso, todo en  $\mathcal{O}(\log N)$ .

# Outline

- 1 Introducción
- 2 Trie
- 3 Hashing
- 4 Función Z**
- 5 Resumen y Consultas

# Función Z

Dado un string  $s$  de tamaño  $n$ , el Arreglo Z (z-array) es un arreglo  $z$  de tamaño  $n$  tal que  $z[i]$  es la longitud del prefijo común más largo entre  $s$  y el sufijo de  $s$  que empieza en el índice  $i$ .

$$z[i] = \text{LCP}(s[0 \dots n-1], s[i \dots n-1])$$

Este arreglo se puede calcular en tiempo y espacio lineal  $\mathcal{O}(n)$ .

|   |    |   |   |   |   |   |   |   |   |   |
|---|----|---|---|---|---|---|---|---|---|---|
| s | a  | b | a | a | c | a | b | a | a | d |
| z | 10 | 0 | 1 | 1 | 0 | 4 | 0 | 1 | 1 | 0 |

Pueden encontrar código de referencia para la Función Z en:

[github.com/.../z\\_function.cpp](https://github.com/.../z_function.cpp)

# Problema 1: String Matching

## Enunciado

Dado un texto  $T$  y un patrón  $P$ , encontrar el número de ocurrencias de  $P$  en  $T$ .

# Problema 1: String Matching

## Enunciado

Dado un texto  $T$  y un patrón  $P$ , encontrar el número de ocurrencias de  $P$  en  $T$ .

Solución con Función Z:

- Construimos un nuevo string combinando el patrón y el texto, separados por un caracter especial que no aparezca en ninguno de los dos (por ejemplo #):

$$S = P + \# + T$$

- Calculamos la Función Z sobre este nuevo string  $S$  en tiempo lineal.
- Si para algún índice  $i > |P|$  se cumple que  $z[i] == |P|$ , entonces encontramos una ocurrencia completa de  $P$  que arranca en esa posición.
- Complejidad total:  $\mathcal{O}(|T| + |P|)$ , óptima y muy limpia de implementar.

## Problema 2: Finding Periods

### CSES 1733 - Finding Periods

Encontrar todas las longitudes que son períodos válidos del string  $S$ . Un período es un prefijo de  $S$  que si se repite concatenándose puede generar todo el string (quizás truncado al final).

## Problema 2: Finding Periods

### CSES 1733 - Finding Periods

Encontrar todas las longitudes que son períodos válidos del string  $S$ . Un período es un prefijo de  $S$  que si se repite concatenándose puede generar todo el string (quizás truncado al final).

Solución con Hashing  $\mathcal{O}(N \log N)$ :

- Para cada longitud  $k$  entre 1 y  $N$ , chequeamos que todos los bloques de tamaño  $k$  sean iguales al primer bloque.
- Para  $k = 1$  comparamos  $N$  bloques, para  $k = 2$  comparamos  $N/2$  bloques, ...
- Hacemos las comparaciones en  $\mathcal{O}(1)$  usando hashes precalculados.
- Complejidad total:  $\mathcal{O}(N + N/2 + \dots + N/N) = \mathcal{O}(N \log N)$ .

# Problema 2: Finding Periods

## CSES 1733 - Finding Periods

Encontrar todas las longitudes que son períodos válidos del string  $S$ . Un período es un prefijo de  $S$  que si se repite concatenándose puede generar todo el string (quizás truncado al final).

Solución lineal con Hashing / Función Z:

- La condición de que el prefijo de tamaño  $k$  sea periodo equivale a decir que el sufijo de tamaño  $N - k$  es idéntico al prefijo de tamaño  $N - k$ .
- Podemos verificar esto usando Hashing en  $\mathcal{O}(1)$  comparando los hashes correspondientes.
- O aún más fácil: usando el arreglo de la Función Z, simplemente verificamos si  $z[k] == N - k$ .
- De esta forma obtenemos la respuesta en  $\mathcal{O}(1)$  por cada longitud  $k$ , logrando una complejidad final óptima de  $\mathcal{O}(N)$ .

# Problema 3: Password

## CF 127D - Password

Encontrar el substring más largo de  $S$  que sea un prefijo válido, un sufijo válido, y que también aparezca al menos una vez más en el medio de  $S$ .

# Problema 3: Password

## CF 127D - Password

Encontrar el substring más largo de  $S$  que sea un prefijo válido, un sufijo válido, y que también aparezca al menos una vez más en el medio de  $S$ .

Solución con Función Z:

- Calculamos el arreglo  $z$  en  $\mathcal{O}(N)$ .
- Si para un índice  $i$  se cumple  $i + z[i] == N$ , significa que el prefijo de longitud  $z[i]$  también es sufijo de  $S$ .
- Además, necesitamos que este prefijo haya aparecido "en el medio". Es decir, tiene que existir algún  $0 < j < i$  tal que  $z[j] \geq z[i]$ .
- Iteramos de izquierda a derecha manteniendo el máximo  $z[j]$  visto hasta el momento y chequeamos estas condiciones en  $\mathcal{O}(1)$  por índice.
- Complejidad total:  $\mathcal{O}(N)$ .

# Outline

- 1 Introducción
- 2 Trie
- 3 Hashing
- 4 Función Z
- 5 Resumen y Consultas**

- Trie: Ideal para queries de prefijos, contar palabras y diccionarios dinámicos. Usado incluso bit a bit (Trie Binario) para problemas de XOR.
- Hashing: Herramienta muy flexible. Permite convertir comparaciones de  $\mathcal{O}(L)$  a  $\mathcal{O}(1)$ . Cuidado con las colisiones. Permite hacer palíndromos dinámicos (con Segment Tree) y comparar strings lexicográficamente en  $\mathcal{O}(\log L)$ .
- Función Z: Extrae toda la información sobre prefijos compartidos en un solo arreglo de tamaño  $N$  en tiempo lineal  $\mathcal{O}(N)$ . Muy útil cuando nos importan patrones periódicos y prefijos-sufijos.

Cualquier duda pueden preguntarme durante esta semana o en cualquier momento a través de:

- Telegram y Codeforces: @lsantire